

JARVIS: Personal Artificial Executive System

A persistent personal executive system: it understands, plans, acts, monitors, remembers, and coordinates across a user's digital and physical environments, while keeping the human in control. This document is the architecture and its decision record. The closing sections — *Considered and Rejected* and *Where This Meets Reality* — are part of the design, not appendices: they record what was deliberately left out and which problems remain unsolved.

Vision

A persistent AI executive assistant that understands, plans, acts, monitors, remembers, and coordinates across the user's digital and physical environments. JARVIS is a continuously operating personal operating system, not a question-answering chatbot.

Core objective: Reduce human operational workload while maintaining human oversight and control.

Security thesis. The defining risk of this system is not a conventional web vulnerability. It is that JARVIS reads attacker-influenceable content (web pages, emails, documents) *and* holds high-consequence capabilities (money, locks, devices) in the same system. Every design decision below is evaluated against one question: *if an attacker controls the text JARVIS is reading, what is the worst thing they can make it do?*

Core Design Principles

Principle 1: Human Remains in Control

JARVIS never assumes authority. Actions are categorized by risk level, and higher-risk actions require progressively stronger authorization.

- **Low-risk:** read files, search web, gather information
- **Medium-risk:** draft emails, create calendar events, edit documents
- **High-risk:** send messages, modify critical files, purchase products, control physical devices
- **Critical:** financial transactions, healthcare decisions, vehicle operation

Confirmation is a real control only if it resists fatigue. A system that asks for a click 40 times a day trains the user to click without reading. See the Authorization Framework for how confirmations are batched, routed out-of-band, and made to show *what* and *from where* — not just *yes/no*.

Principle 2: Plan Before Acting

Every request follows: **Intent** → **Plan** → **Review** → **Execute** → **Verify**. The assistant never jumps from request to action.

The plan is reviewed against its trigger source. If a plan to move money or message someone originated from content JARVIS *read* (rather than from a direct user instruction), that provenance travels with the plan and is surfaced at the Review step.

Principle 3: Continuous World Model

JARVIS maintains an internal representation of user state (preferences, habits, skills, projects, goals, relationships), digital state (accounts, devices, services, files, servers), and physical state (smart home, sensors, cameras, robots, vehicles).

The world model is a write target, therefore an attack target. A false "fact" written once is trusted indefinitely. Every entry carries provenance (who/what wrote it, from what source). Untrusted-source content never writes directly to durable semantic memory without passing review.

Principle 4: Verify Everything

Every action is independently checked. No agent verifies its own work.

Independence must be decorrelated, not just separate. Two agents that both read the same poisoned web page and "agree" are not a verification — they are a single point of failure wearing two hats. The verifier checks against **ground-truth state** (did the money actually move, did the file actually change) and must **not** consume the same untrusted input that drove the action.

Principle 5: Capability and Exposure Are Separated

The component that reads untrusted content is never the component that holds high-risk capabilities. Reading the web and moving money happen in different trust boundaries that communicate through a narrow, typed, reviewed interface. This is the single most important structural decision in the system.

Principle 6: Untrusted Content Is Data, Never Instruction

Anything JARVIS reads — web pages, emails, file contents, document text, tool outputs — is data. It can never escalate into an instruction that changes JARVIS's goals or permissions. This is enforced structurally (separate channels, typed boundaries), not by prompting the model to "ignore injection attempts," which is not a security control.

Principle 7: Default-Safe on Ambiguity

On timeout, ambiguity, conflicting signals, or authorization failure, the system defaults to **no action**, not best-guess. A wrong guess at Level 3+ is worse than inaction.

System Architecture

Surfaces Are Detachable Views Over Host-Resident State

A cross-cutting property that governs every layer below. JARVIS runs on a host (a laptop or desktop); **all durable state — projects, memory, world model, skills, authorization — lives on the host, never on a surface**. A *surface* is any device a user interacts through: the host's own screen, a phone, or an optional peripheral. Surfaces are **views** that attach to host state, render it, forward input back, and detach without the system caring.

Consequences, stated so they are not re-derived later:

- The host's own screen is the default surface and is always available.
- Any additional surface is optional. It may be absent for weeks; its absence is a non-event, never an error, and never blocks the system.
- A surface holds **no state it can lose**. It renders the current view and forwards touches/clicks/speech. If it vanishes mid-session, the host is unaffected; when it reattaches, it picks up the current view.
- The daily driver (host) must never depend on an occasional peripheral. The dependency runs one way only: surfaces depend on the host, never the reverse.

This is what lets a second surface be added later (or rolled up and put in a drawer) without touching the architecture. The Holomat below is the first such optional surface.

Layer 1: Conversational Layer

Natural interaction: voice and text conversations, multimodal understanding, emotional awareness, clarification requests.

- **Speech System:** speech-to-text, text-to-speech, wake-word detection
- **Dialogue Engine:** maintains conversation state, tracks goals, handles interruptions

Trust note. Direct user voice/text is the *only* channel that can issue instructions. It carries the highest trust label. Everything ingested by other layers is data.

Layer 2: Agentic Layer

Thinking and planning. Specialized agents:

- **Planner Agent** — creates execution plans
- **Research Agent** — web research, fact verification, information gathering
- **Project Manager Agent** — deadlines, milestones, dependencies, project graphs
- **Memory Agent** — stores and retrieves knowledge
- **Scheduling Agent** — calendar, time blocking, availability
- **Safety Agent** — risk, permissions, compliance
- **Auditor Agent** — verifies outcomes
- **Introspection Agent** — tracks per-component success rates, confidence calibration, failed-plan and unreliable-tool stats. **Surfaces** these to the user and the Governor; it does **not** silently auto-route around weak components. A component at 72% success may still be the only one that can do a task, so rerouting is a recommendation requiring confirmation, never an automatic override.

Trust zoning of agents. Agents split into two zones:

- **Untrusted-read zone:** Research Agent, inbound Communication, file-read. These touch attacker-influenceable content. They have *no* direct access to Level 3+ capabilities.
- **Privileged-action zone:** the executors that can message, spend, or control devices. They receive only typed, validated requests from the untrusted zone — never raw content, never raw instructions.

The Safety Agent and Auditor Agent sit at the boundary and must not ingest raw untrusted content when evaluating an action derived from it.

This raises an obvious objection: a verifier forbidden from reading the content that drove an action seems unable to judge the action. "Transfer \$400 to account X" is meaningless without the why, and the why is the poisoned content. The resolution is that the verifier never checks the rationale. It checks the **typed request** against **ground-truth state**. The untrusted zone does not hand the executor "the web said to pay this invoice." It hands a typed object carrying the action, the amount, the destination, the provenance label, and the trigger source. The verifier checks that object against state (does the destination match a known payee, is the amount under the cap, has the user ever approved this payee) and against the diff shown at the confirmation gate. The rationale is deliberately left on the untrusted side because it is not what is being verified. The claim is, against state. One consequence for the build: the typed inter-zone request schema becomes a security-critical artifact. It must carry everything verification needs and nothing that requires interpreting prose. The day the verifier seems to need the rationale to make a call is the day the boundary type is revealed as underspecified, not the day the verifier should be allowed to read further.

Layer 3: Operating System Layer

Bridge between reasoning and execution.

Runtime Governor. Supervises all agent activity at runtime, and is deliberately the dumbest privileged component in the system: it counts, it pattern-matches, and it listens. It owns three mechanisms, not one. First, a **budget meter** against rolling ceilings (actions/hour, spend/day, outbound messages/hour, API retries), keyed globally and per-project. The per-project resource ceiling is not a separate mechanism, it is this same meter with a project-scoped bucket. Second, a **loop detector** over recent action history (same tool and arguments repeated, or short $A \rightarrow B \rightarrow A \rightarrow B$ cycles), which fires independently of budget because a tight retry loop on a free local tool burns no money and trips no spend ceiling. Budgets catch expensive, loops catch repetitive, and the two failure modes do not overlap. Third, a **progress trigger**: terminate when no objective completes in N actions. This last one cannot be computed inside the Governor, and that constraint is load-bearing. "Objective completed" is defined by the goal and project object, so the Governor does not inspect agent output to guess whether progress happened. It subscribes to a typed "objective satisfied" event the project object emits, and watches the gap between such events. The alternative, a supervisor that reads agent output to judge progress, would pull attacker-influenceable reasoning back into the kill path, which is exactly the correlation Principle 4 removes. A dumb supervisor is a trustworthy supervisor: it never interprets prose, so it cannot be talked out of killing a runaway. This Governor is the single collection point for the autonomy budgets specified under Long-Term Autonomy; they live here because they share a kill path, not because their logic is fused.

- **Tool Router** — which tools, what order, fallbacks. **Default-deny allowlist.** The router can only invoke explicitly permitted tools. Each tool declares a maximum blast radius. Define what JARVIS can *never* do, independent of authorization level.

Ordering contract: provenance pre-filters, it does not post-check. The provenance gate and the Tool Router have a strict ordering, and getting it wrong silently voids the security model. Provenance is a typed field on the request from the moment of ingestion, and the router's allowlist is parameterized by it. An untrusted-derived request does not get the full tool menu and then get escalated after the fact. It gets a smaller menu, the Level ≤ 2 tools, before the router runs at all. Escalation is therefore a pre-filter on what is selectable, never a post-hoc level bump on an action whose shape was already chosen under attacker influence. The distinction is structural: "check the action after building it" relies on the human catching a bad diff at the gate, which Principle 1 forbids depending on. "Constrain what can be built" is the actual enforcement. The build consequence is that provenance is data the router reads, not metadata the Safety Agent inspects later.

- **File System Agent** — create/edit/organize files. **Hard-excluded paths:** credential vault, `~/ .ssh`, system directories, JARVIS's own audit log and config. Non-overridable.

- **Browser Agent** — DOM interaction, form completion, navigation, extraction (vision / accessibility / API modes). **Highest-risk component in the system:** it both reads untrusted content and can fill forms. Credentials are injected at the execution boundary; the agent never receives raw tokens in context. Screenshots are scrubbed of credential-bearing fields before storage.
- **Communication Agent** — email, SMS, Slack, Teams, Discord. **Inbound is untrusted data. Outbound is a Level 3 action** requiring confirmation. Inbound content can never auto-trigger outbound.
- **Device Agent** — PCs, phones, tablets, servers, **and fabrication machines (3D printers, laser cutters)**. The common case for these is not general control but a single first-class verb: **export — send this finished artifact to that machine**. Design surfaces (e.g. the Holomat) originate the export; the Device Agent governs the dispatch under normal authorization and reliability handling. One-click for the user, fully gated underneath.
- **Cloud Agent** — AWS, Azure, GCP, databases
- **Smart Home Agent** — lights, thermostats, cameras, locks, appliances. **Locks/cameras are Level 4** (physical safety). Never controllable by an action derived from untrusted content.

Layer 4: Embodied Layer

Physical-environment interaction: vision system (object detection, scene understanding, OCR, optional face recognition), hand/gesture tracking, audio perception (sound localization, speaker ID, event detection), robotics interface (mobile robots, arms, drones), spatial mapping.

Embodied actuation is Level 4 minimum. Any physical movement near people has a hard rate/force ceiling and a hardware kill path independent of software.

Optional Surface: The Holomat

The Holomat is the first **optional surface** (see *Surfaces Are Detachable Views*): a roll-up, projected, touch-sensitive working mat for design and fabrication. It is a peripheral, not part of the core system — JARVIS runs fully without it, and it may sit unused for long stretches. It is documented here at build-recipe depth specifically so a future build does not have to reconstruct how it works from scratch.

What it is and is not

- **Is:** a large physical surface you can draw on, lay material on, and design against at life size, with a projected JARVIS interface on top and capacitive touch underneath. A second view onto host-resident project state, plus a one-tap path to send finished designs to shop machines.
- **Is not:** a screen (you can cut and work on top of it without damage), and not a store of state. It renders the host's current view and forwards touch. Unplugged, it is inert; the host does not notice.

Why it earns a place

It exercises three parts of the architecture on cheap, physical, immediately- falsifiable ground: the **app/skill model** (each tool is one app on the surface), the **project object** (a design session is a project with material, tasks, and an export target), and **export-to-device** as a first-class verb. It does *not* exercise the security spine, autonomy, or memory machinery — those stay on the host and the mat never touches them. Treat it as a design-and- dispatch surface, not as "JARVIS made cinematic." The hand-waving HUD theatrics are the seductive wrong target; reliable export to a real machine is the useful one.

Physical build (three bonded layers)

Bottom to top — this stack is the part most likely to be forgotten:

1. **Rubber base sheet.** Protects the workbench and stops the mat sliding.
2. **USB capacitive touch foil.** Self-adhesive backing sticks to the rubber; plugs in over USB and reports touch coordinates. This is the input layer.
3. **Matte black vinyl top.** The projection surface and finish.

The hard part is the vinyl, not the electronics. Adhering the vinyl bubble-free is the entire difficulty of the build. Work in sections with a straight edge, a little at a time; expect multiple attempts. Once adhered it is extremely hard to peel back, so commit each section deliberately. Trim excess with a razor after it is down. The electronics are forgiving; the lamination is not.

After bonding, all three layers roll up together. Hide the touch-foil controller and cable in a small 3D-printed enclosure at one edge for a clean finish.

Projection + compute

- **Projector** mounted overhead (ceiling bracket) or on a stand, throwing the interface down onto the mat.
- **Raspberry Pi 5** (or any modest computer) drives the projector over HDMI and reads the touch foil over USB. Note: the Pi here is a *thin client / surface driver*, not the JARVIS host. It renders the view JARVIS serves and forwards touch; it holds no project state. JARVIS itself runs on the laptop/desktop host.

Software (the part that is actually JARVIS)

The on-mat interface is a JARVIS view: the app carousel and the project surface, rendered for touch. Apps follow the standard skill/app authoring model — **one JSX file per app**, dynamically added to the carousel, documented by a published authoring guide (see *Skill Registry*). Representative apps worth having: a 2D tracing/scaling tool (place real material, trace at life size, cut), a freehand drawing tool, a 3D model viewer, a voice-to-3D model creator, and per-machine dispatch apps (3D printer status + G-code, laser-cutter export).

Export-to-device

Finished designs leave the mat by one explicit action: **export** → **target machine**. Because the surface designs at life size, scale is correct on the first send. Export targets are the host's Device Agents (3D printers, laser cutters); the dispatch itself runs through the host's normal device-control path, **including its authorization and reliability handling** — the mat originates the request, the host governs it. A failed print, a disconnected machine, or a mis-scaled cut is the host's problem to surface and recover, not the mat's. (This is exactly the device-control reliability named under *Where This Meets Reality*; the mat does not get to pretend it is solved.)

The thin-client seam (where the surface contract gets tested)

The surface contract says the host owns all state and the mat renders a view and forwards input. For a display that is enough. For a surface whose headline verb is export-to-machine, "render a view and forward touch" hides the one question the rest of the section depends on: what travels between host and mat, and what happens when they disagree.

The mat forwards intent, not pixels-plus-clicks. A naive thin client streams the host's framebuffer down and raw touch coordinates up. That arrangement has a fatal property for a dispatch surface: the mat can show a frame the host has already moved past, the user taps what they see, and the coordinate lands on whatever now occupies that pixel. A stale frame on a media player is a cosmetic glitch. A stale frame on the laser-cutter dispatch app is a touch that confirms an export the user never looked at. So the mat does not forward "touch at (x,y)." It forwards "user activated control C," where C is an identified element of the view the host served, carrying the view-version it belongs to. The host rejects any input tagged with a view-version it has already superseded. A touch against a stale frame fails closed and the user retaps the current one. This is the Default-Safe-on-Ambiguity principle applied at the surface boundary: a touch whose context the host can no longer confirm resolves to no action, not best-guess.

Export is never a single forwarded touch. Even with view-versioning, the export verb does not fire from one tap on the mat. Export is a Level 3 dispatch to a physical machine, and it routes through the host's normal authorization and reliability path exactly as the rest of the section insists. The tap on the mat originates the request. The confirmation happens where Level 3 confirmations always happen, out-of-band, on a surface that is not the one that requested the action. That the requesting surface is a projected mat rather than a browser changes nothing: an attacker or an accident that drives the mat still cannot

complete an export, because completion lives on the host's out-of-band gate, not on the mat. The mat cannot approve what the mat requests.

Latency is a correctness property here, not a comfort one. The acceptable touch-to-render budget for a drawing tool is whatever feels responsive. The acceptable budget for the dispatch apps is bounded by something stricter: the view-version round trip has to be fast enough that the user is reliably acting on the current frame, or the fail-closed retaps above turn from a rare safety net into the normal interaction and the surface feels broken. This is the real reason the Pi is a thin client and not a peer. A thin client that forwards identified-intent against a versioned view can be made correct over a flaky link by failing closed. A fat client that holds its own copy of project state and reconciles later cannot, because now there are two states that can diverge, and the section's entire "holds no state it can lose" guarantee is gone. The thinness is not a cost-saving measure. It is what keeps the surface contract true under a real network.

What this does not solve. None of the above makes device control itself reliable. A view-versioned, fail-closed, out-of-band-confirmed export still dispatches to a laser cutter that can jam, a printer that can be offline, a cut that can be mis-scaled by a bad model the user approved in good faith. Those are the host's Device Agent problems, named under Where This Meets Reality, and the mat does not get to pretend they are solved because it dispatched cleanly. The seam work here guarantees the *right* artifact is dispatched with the *user's* confirmation. It guarantees nothing about what the machine then does with it.

Attach / detach behavior

The Holomat obeys the surface contract without exception: it attaches to the host, renders the current view, forwards input, and detaches cleanly. Rolling it up mid-session loses nothing; reattaching resumes the current view. Nothing in the daily-driver path may depend on the mat being present.

Memory Architecture

- **Working Memory** — seconds to hours: current tasks, active conversations
- **Episodic Memory** — events, interactions, experiences (meeting summaries, completed tasks)
- **Semantic Memory** — facts (user prefers Python, favorite restaurants, schedules)
- **Procedural Memory** — workflows (application workflow, paper workflow, deployment workflow)

Memory security model:

- Every memory carries **provenance** (source + trust label + timestamp).
- Untrusted-source content cannot write to **semantic** or **procedural** memory without review — these are the durable, trusted stores.

- Credential-bearing data is **never** written to any memory tier; it lives only in the vault.
- Episodic memory storing screenshots/logs is **scrubbed** of secrets at write time.

Fact Lifecycle. Provenance says *where* a fact came from; it does not say *when it goes stale*. Every semantic fact additionally carries an **expiration policy** keyed to fact-type:

- Volatile ("user works at X", "current project deadline") — expires after a verification interval; flagged stale until reconfirmed.
- Stable ("user's birth year", "user prefers Python") — effectively non-expiring.

Without expiration, semantic memory degrades silently: a true fact rots into a confidently-held false one. Cheapest high-value control in this design — it is a schema field plus a staleness check, not a new subsystem.

Skill Registry (extends procedural memory). Workflows accumulate as reusable *skills*, so capability compounds rather than being re-derived each time. Each skill declares: trigger conditions, required tools, required permissions, success criteria, rollback procedure.

Authoring artifact (low barrier on purpose). A skill/app has a *defined authoring format* and a *published authoring guide*, so the system is extensible by the user, not only by its builder. The concrete form: **one JSX file per app** plus a markdown authoring doc, dynamically registered into the interface (e.g. the Holomat's app carousel) as it is created. Keeping the authoring unit small and documented is what turns the registry from a builder-only internal into a surface the user actually extends.

Security seam — do not miss this. A skill with stored permissions and an auto-submit step is a *pre-authorized action template*. If a skill can be *triggered* by untrusted content (e.g. an "apply to this job URL" instruction injected into a web page), it becomes a clean bypass of the confirmation gate. Therefore: skill **trigger conditions** route through the same **provenance gate** (Authorization Framework) as any other action. A skill triggered by untrusted-derived input is escalated and capped exactly like a raw action. Skills never carry standing authorization that survives the provenance check.

World Model

A continuously updated digital twin: People Graph, Project Graph, Device Graph, Knowledge Graph.

Each node/edge carries provenance and a confidence score. A claim sourced from untrusted content is flagged as such and cannot silently harden into a trusted fact.

Identity confidence (People Graph field, not a new layer). "Professor Aryal" and "someone claiming to be Professor Aryal" are different objects, and the system must not conflate them. Identity is distinct from authorization: authorization asks *what is this actor allowed to do*; identity asks *is this actor who they claim to be, and how sure are we*. Each

People Graph node therefore carries an **identity-confidence** value and the **authentication basis** for it:

email-verified | voice-verified | device-verified | MFA-verified | unverified

This feeds the trust machinery already running throughout the design: a Level 3 action requested by an unverified inbound identity is treated as untrusted-derived and gated accordingly. No new "Identity Layer" is needed — identity confidence is a property on a graph that already exists, wired into the provenance gate that already exists. The novelty proposed by the review was real; the right size for it is a field, not a stratum.

User focus mode — declared, not inferred. One tempting direction is to model the user's *emotional state* (overwhelmed, stressed, burned out, focused) so JARVIS could change behaviour accordingly. This is declined in its inferred form and reduced to a declared one. Two reasons. First, inferring an internal emotional state from digital signals (email cadence, late-night activity) is unreliable and slightly invasive — a clinical-ish judgment the system is not positioned to make, and wrong in either direction it harms (treating a focused person as fragile, or missing real distress). Second, "silently reshape what I show the user based on my read of their fragility" is a manipulation surface, not a feature: it puts the system in the position of deciding when to withhold things for the user's own good. The honest, smaller version of the real need: a **user-set focus mode** ("I'm slammed this week — surface only what's urgent") that the user controls explicitly. That is a single declared field on the user model gating attention and notification, not an inferred emotional model. The refusal is the valuable part; it is recorded in *Considered and Rejected*.

Projects: The Organizing Object

The central organizing decision. A naive design runs *User* → *Conversation* → *Agents* → *Tools*, with memory, scheduling, deadlines, and decisions scattered across layers. That mirrors how the *machine* is built, not how a person *delegates*. Nobody says "use the browser agent." They say "help me publish the paper," "help me get the internship," "help me build VitaDeck."

So **Projects become the persistent object that everything else hangs off**. Agents operate *on* projects; they are not the top-level unit. A project ties together what would otherwise fragment into isolated tasks:

Project: Publish JBHI paper

└ Lifecycle (candidate | active | completed | archived | abandoned)

└ Attention (focus | background | dormant — only for active projects)

- └─ Goals (priority, status, success criteria, deadline, dependencies, provenance, review cadence)
- └─ Tasks (the actionable breakdown)
- └─ Skills (reusable workflows scoped to this project)
- └─ Memory (episodic + semantic entries tagged to this project)
- └─ Temporal state (deadlines, waiting states, recurring + external-event subscriptions)
- └─ Decisions (what was decided, why, on what evidence, at what confidence)
- └─ Resources (budget/attention this project is permitted to consume)

A naive design would treat Goal Lifecycle, Decision Ledger, Temporal state, and Reconciliation as four separate subsystems. They are not separate — they are **facets of the project object**, and folding them in is what keeps this architecture converged rather than accreting boxes. Each facet:

Goals (first-class, not implicit). A goal carries priority, owner, status, success criteria, deadline, dependencies, provenance, and a review cadence. Without first-class goals, planning is reactive — it responds to the next request instead of advancing a standing objective. Dependencies between goals are what let the Planner sequence work strategically ("experiments complete → figures complete → submit") rather than greedily. The goal object is also where "objective completed" is defined, and it emits a typed satisfaction event the Runtime Governor subscribes to for its progress trigger (see *Layer 3*). The Governor never computes completion itself; the goal object declares it.

Decisions (recorded, not re-litigated). Distinct from the action audit trail: the audit trail records *what was done*, the decision record records *what was chosen and why* — including choices **not** to act. "Did not apply to internship — missing prerequisite — evidence: job posting — confidence 0.88." This stops the system from reconsidering the same settled question every cycle. It is goal-state-with-rationale, which is why it lives on the project, not in a separate ledger.

Temporal state (waiting is a first-class status). Much of what reads as "executive function" is really temporal reasoning, and the case current schedulers handle worst is the *waiting state*: "awaiting professor response — recheck in 72h." A project tracks deadlines, recurring obligations, future commitments, and open waits. This is a scheduler primitive (wait-conditions), not a new layer, but it is load-bearing enough to be an explicit project facet rather than an implicit detail.

Reconciliation (active, not just passive expiry). Fact Lifecycle (see *Memory Architecture*) flags a fact as *stale*; reconciliation actively *re-checks* it on a cadence — "does the user still work here? is this device still online? does this project still exist?" This is one behavior added to the lifecycle, not a sixth subsystem. Project-scoped facts get project-scoped reconciliation cadences. Derived facts additionally record *what they were derived from*, so that when a

source fact is reconciled false, reconciliation can cascade to the facts that depended on it. This is a one-line strengthening of existing provenance — not a full belief-dependency graph, which is declined as out of scope (see *Considered and Rejected*).

Resources (per-project budgets). Each project carries a ceiling on the API/compute/attention it may consume, enforced by the Runtime Governor. This is the *prevention* floor. A cross-project *optimizing* allocator — deciding which project should get scarce resources when several compete — is a genuinely later concern (see *Considered and Rejected* → *Deferred*); it requires measured contention a single-user system will not have for a long time.

Conflict resolution. When signals collide — calendar says attend, project says finish, smart-home says leave, user says stay — the tie-break is mostly already implicit in the trust model: **direct user instruction carries the highest trust label and wins**. The residual case (project A vs project B when the user has not spoken) resolves on the **priority** field of the goal/ project object plus a small ordered rule set. This is a priority comparison, not a standalone "policy engine"; naming it as a concern is worth doing, but building it as a subsystem before two projects actually contend would be speculative.

Lifecycle (orthogonal to attention). A project exists on a *lifecycle* axis — **candidate** → **active** → **completed** → **archived** → **abandoned** — that is separate from the *attention* axis below. A single notion of "status" would blur these two: "dormant" would do double duty as both "not currently worked" and "barely alive," which is a real seam. They are kept separate:

- **Lifecycle** answers *does this project exist and is it live or finished*. Only **active** projects carry an attention state. A **candidate** is a potential project not yet committed to (see environmental events below); **completed**, **archived**, and **abandoned** are terminal and consume no attention by definition.
- **Attention** answers *how much focus a live project gets right now*.

The two axes are independent: an **active** project can be **focus**, **background**, or **dormant**; a **candidate** has no attention state until promoted to **active**. Keeping them orthogonal is the fix — not a new object, just two fields that must not be allowed to overlap.

Attention (what the system deliberately ignores). A system that *acts* but never decides what to *ignore* treats all thirty projects as equally live and drowns. Against finite compute and finite user attention, that capacity to ignore must be represented explicitly. It is not a new layer: it is an **attention status on the (active) project object**, with three states and escalation rules.

Focus actively worked; consumes real compute + user attention

Background monitored cheaply; reconciliation runs, no active work

Dormant no attention; only an escalation trigger can wake it

Escalation rules pull a project back into focus on a condition, and they reuse the **temporal-trigger machinery already in the temporal facet** — there is no new mechanism here:

Paper project: active / dormant

└ trigger: reviewer responds → promote to active / focus

This mirrors how senior people actually operate: not by managing thousands of tasks, but by managing which few are allowed to occupy attention at all. The Runtime Governor enforces that *focus* projects are the ones permitted to spend; *background* and *dormant* cost almost nothing. Attention is therefore also a cost-control mechanism, not only an organizing one.

Environmental events (opportunities) feed the same machinery. Not every trigger comes from inside a project's own clock. Some are *environmental*: a scholarship opens, a relevant CFP is released, a stocked item becomes available, a contact replies after three months. One option would be an "Opportunity Sources" object type in the world model to hold these. That over-separates them. The right home is the escalation machinery that already exists — the temporal facet simply also **subscribes to external event sources**, not only internal project clocks. The escalation rule "reviewer responds → promote project" and the rule "CFP for a relevant venue released → surface a **candidate** project" are the same mechanism pointed outward. An opportunity is precisely *what becomes a candidate project*, which is why this folds into the lifecycle axis above rather than standing alone. One mechanism, two directions (internal clock, external feed); no new object.

Candidates are discarded by default. The spec above says how candidates are *created*; it must also say how most are *killed*, or the system becomes a machine that perpetually proposes work. Real life generates far more opportunities than anyone can pursue, so the default for a candidate is **discard, not surface**. A candidate is presented to the user only if it clears a user-set relevance bar (matched venue, matched topic, explicit standing interest); unpursued candidates **expire** on a short cadence rather than accumulating. Promotion **candidate → active** is always a user act, never automatic. This is a default and an expiry, not a filtering subsystem — the burden of proof is on the candidate to survive, not on the user to dismiss it.

On a Utility Model — explicitly declined. A natural-seeming move is a formal objective function scoring each project on priority × deadline-pressure × expected-impact × effort × uncertainty × risk, emitting an expected-value number to drive prioritization. This is rejected on purpose. Inventing six estimates per project and multiplying them manufactures false precision: the output looks rigorous while resting entirely on guesses, and it would resolve genuine prioritization tension *silently*, inside a formula, exactly where the system should instead surface the tension to the user. The honest design: the user sets priority, deadlines generate pressure, attention states gate spend, and when projects genuinely contend for the same scarce resource the system **asks** rather than scoring. Declining to formalize prioritization into a scalar is the more sophisticated choice, not the less complete one.

Authorization Framework

Level	Scope	Confirmation
0	Read-only	None
1	Drafting / recommendations	None
2	Non-destructive actions	Single confirmation
3	Communication / publishing	Strong confirmation
4	Financial / physical actions	Multi-factor
5	Emergency override	Explicit predefined permissions

Anti-fatigue confirmation design:

- **Level 2** routine actions are **batched** into a single periodic review, not interrupt-per-action.
- **Level 4** confirmations go **out-of-band** — a push to your phone, never the same interface that requested the action. An attacker controlling the screen can't also approve from it.
- **Every confirmation shows DIFF and SOURCE:** what exactly will change, and what triggered it. "This transfer was triggered by content from `<url>` — proceed?" An action whose trigger is untrusted content is flagged red regardless of level.

Provenance gate (overrides the table above). Any action *derived from untrusted content* is escalated by at least one level and can never exceed Level 2 automatically, no matter what the content claims. Permission asserted inside read content ("the user authorized this") is void. The gate is enforced as a pre-filter on tool selection, not a post-hoc check on a built action (see *Layer 3 → Ordering contract*): an untrusted-derived request never sees the Level 3+ tool menu in the first place.

Hard ceilings (non-overridable by any level):

- No transfer above a user-set cap without out-of-band MFA.

- No lock/camera/vehicle control from an untrusted-derived action, ever.
- No write to vault, audit log, or system config by any agent.

Self-modification is not exempt. JARVIS will eventually want to change its own workflows, prompts, skills, memory structures, and tools — for example, replacing a 96%-success skill with a 99% version. This does not require a separate "constitutional layer." A self-modification *is an action*, and it routes through this same framework: changes to skills, prompts, or tools are high-level actions requiring confirmation, and a self-modification *triggered by untrusted content* hits the provenance gate like anything else. Autonomy never includes the standing right to silently rewrite the rules it operates under. The authorization framework governs this; stating it explicitly closes the gap on paper rather than leaving it assumed.

Verification Framework

Every action produces an **Execution Log** (what happened), **Evidence** (screenshots, API responses, receipts — secret-scrubbed), a **Confidence Score**, and a **Recovery Plan** (how to undo).

Verifier independence:

- The Auditor checks against **ground-truth state**, not the planner's narrative of success.
- The Auditor does **not** read the untrusted content that drove the action, preventing correlated compromise. It checks the typed inter-zone request against state instead of the prose rationale (see *Layer 2* → *Trust zoning*).
- For irreversible actions (sent message, completed payment), verification happens **before** execution at the confirmation gate, because there is no meaningful "undo."

Dry-run / diff preview (the achievable slice of "simulation"). Before a Level 3+ action, the system produces the concrete **diff** the action would cause — recipient and full text of an outbound message, exact amount and destination of a transfer, files a deploy would change — and shows it at the confirmation gate. This is *not* a "digital twin that estimates downstream effects": predicting what sending a given email actually causes in the world is a research problem, not a layer, and is explicitly out of scope. The diff preview is deterministic, buildable, and covers the case that matters — the user seeing exactly what will happen before it happens.

Long-Term Autonomy

- **Reactive Mode** — responds to user requests

- **Proactive Mode** — initiates recommendations (deadline reminders, system alerts, opportunity discovery)

Autonomy governors. Proactive mode operates under hard budgets: max actions/hour, max spend/day, max outbound messages/hour, max API retries. These are the budget meter of the Runtime Governor (see *Layer 3*), collected there because they share its kill path. A runaway or hijacked loop hits a ceiling, not your bank account. Proactive mode can *recommend* Level 3+ actions but cannot *execute* them without the normal confirmation gate.

Predictive Intelligence

Future capabilities: detect project risks, anticipate scheduling conflicts, suggest resource allocation, recommend actions before requests occur.

Example: "Your manuscript deadline is approaching. Statistical analysis remains incomplete."

Predictions are recommendations only. A prediction never auto-arms an action.

Security Architecture

Standard application baseline

Apply to every service JARVIS exposes (this is the Hackathon Playbook p.11 checklist, and it is necessary but *not sufficient* here):

- Scan codebase for hardcoded keys/tokens/passwords; all secrets in env vars or vault; nothing sensitive in frontend or git.
- Rate-limit **all** endpoints; max 5 auth attempts / 15 min.
- HTTPS/TLS on anything handling user data.
- Sanitize and size-limit all inputs.
- Strip console errors, debug logs, placeholder text.
- OWASP Top 10 pass, documented.
- Honest "known issues / limitations" section.

Sandboxed Execution

Separate environments for browser, code execution, device control, file operations. These sandboxes are also the **trust boundaries** of Principle 5 — the browser/research sandbox cannot call the financial/device executors directly.

Credential Vault

Stores API keys, passwords, tokens — encrypted and isolated. **Injection at the execution boundary:** the orchestrating model never receives raw credentials in its context. The vault fills them into the final request at the last hop. The agent reasoning about the task sees a placeholder, not the secret.

Audit Trail

Permanent record of decisions, actions, authorizations. **Tamper-evident and externally writable:** the log is append-only and written by a component JARVIS **cannot modify**. An agent that can edit its own audit log has no audit log.

Agent-specific controls (absent from the source checklist)

1. **Instruction/data separation** — untrusted content cannot become instruction; enforced structurally.
 2. **Trust-boundary isolation** — read-untrusted surface separated from high-capability surface (Principle 5).
 3. **Tool allowlist + per-tool blast-radius ceiling.**
 4. **Autonomy budget governors** — caps on actions, spend, messages, retries.
 5. **Credential non-exposure** — model never sees raw secrets.
 6. **Fatigue-resistant, out-of-band, source-revealing confirmations.**
 7. **Decorrelated verification** — auditor avoids the same untrusted input.
 8. **Memory-poisoning defense** — provenance tags, no untrusted writes to durable memory.
 9. **Kill switch** — single command halts all agents; hardware kill for embodied layer.
 10. **Default-safe on failure** — ambiguity resolves to no action.
-

Suggested Development Roadmap

Each phase carries the security controls relevant to the capabilities it unlocks. Capability does not ship ahead of its controls.

- **Phase 1 — Personal AI Assistant** Conversation, memory, tool calling, browser control. *Security gate:* instruction/data separation, credential non-exposure, tool allowlist, memory provenance. (Browser control is already high-risk — these are not deferrable.) *Also ship here:* Fact Lifecycle (expiration) — it is nearly free and prevents memory rot from day one.

- **Phase 2 — Executive Agent** Planning, project management, scheduling, research workflows. *Security gate*: trust-boundary isolation between research-read and any action executor; decorrelated auditor; provenance-as-pre-filter ordering contract on the Tool Router. *Also ship here*: Runtime Governor (budget meter, loop detector, and the progress trigger wired to the goal object's satisfaction event); Skill Registry with provenance-gated triggers; Introspection Agent (surface-only).
 - **Phase 3 — Personal Operating System** Device control, smart home, continuous world model. *Security gate*: Level 4 hard ceilings on locks/cameras; provenance gate enforced; autonomy budget governors live.
 - **Phase 4 — Embodied Intelligence** Vision, gesture control, robotics interfaces. *Security gate*: hardware kill path; force/rate ceilings; no untrusted-derived physical actuation.
 - **Phase 5 — Autonomous Executive** Predictive assistance, cross-system orchestration, long-horizon planning. *Security gate*: full audit-trail tamper-evidence; out-of-band MFA on all Level 4; red-team pass on prompt injection across every read surface.
-

Success Metric

The test is not whether JARVIS can answer questions. It is:

Can the user delegate an entire project and receive a verified, auditable, high-quality result with minimal intervention while maintaining trust and control

—

— *and* can a motivated attacker who controls a web page, an email, or a document JARVIS reads gain no high-consequence capability they did not already have? If the answer to the second clause is no, the first clause does not matter.

Considered and Rejected

Decision-log discipline: paths considered, declined, with the reason. Recorded so a future me does not re-litigate them from scratch.

Rejected: three-computer split (Brain / Hands / Vault)

Proposal: physically/logically split the system into Brain (reasoning, planning, memory), Hands (browser, files, devices, execution), and Vault (credentials, identity, authorization), such that Brain never sees secrets, Hands never owns secrets, Vault never reasons.

Why rejected as the primary boundary: it draws the split along the wrong axis. The boundary that actually contains prompt injection — and the one this plan is built on (Principle 5) — is *exposure to untrusted content vs high-risk capability*. The Brain/Hands/Vault axis is *reasoning vs execution vs secrets*, which cuts through the wrong place:

- The **Research Agent** reads the fully-untrusted web but lives in "Brain" next to the Planner.
- The **Browser Agent** reads the untrusted web *and* fills forms, living in "Hands."

So untrusted content enters in **both** Brain and Hands. The split does not contain it. "Brain never sees secrets" is real but already covered by credential non-exposure (Security Architecture), and it is the minor half of the problem. Adopting this split *instead of* the trust-zone split would trade the load-bearing boundary for a cosmetic one.

Disposition: the Brain/Hands/Vault separation is fine as a *secondary* organizing layer on top of the trust-zone boundary, never as a replacement for it. Vault isolation (already present) is kept; the rest is not promoted to primary architecture.

Deferred (right idea, wrong time for a personal prototype)

- **Event Bus.** Loose coupling and replay are real benefits, but this is an implementation choice adopted when direct calls start hurting — not speculative scaffolding for scale a single-user prototype does not have. Revisit when agent-to-agent calls become a debugging problem.
- **Full Simulation / "digital twin" of downstream effects.** The achievable slice (deterministic diff preview) is adopted now under the Verification Framework. Estimating real-world consequences of an action (what does sending *this* email cause?) is a research project, not a layer. Deferred indefinitely.
- **Multi-User / Identity Graph.** Correct that a *product* needs it, but the necessity is scoped to enterprise/multi-party use. This is a personal system. Deferred until JARVIS graduates out of "personal" into a real multi-party entity — by the pipeline's own rules, that is the point at which it stops being this project.
- **Cross-project resource allocator.** Per-project budgets (prevention) are in. An *optimizing* allocator that arbitrates scarce API/compute/attention across competing projects is deferred — it requires measured contention a single-user system will not produce for a long time. The Runtime Governor already prevents abuse; optimization on top of prevention is a problem one would be fortunate to have.

- **Standalone Policy / Conflict Engine.** Folded into the project object as a priority field plus an ordered rule set (see *Projects* → *Conflict resolution*). Not built as a separate arbitration subsystem until two projects actually contend in practice.

Right concern, wrong altitude (absorbed, not added as layers)

A class of proposals recurs: four new top-level "primitives." Two named real gaps and were absorbed as fields on existing objects (Attention → project status; Identity → People Graph field). The rest are recorded here as concerns the existing architecture already covers, deliberately *not* promoted to layers:

- **Utility Model — rejected outright** (not merely deferred). A scalar expected-value score over priority × deadline × impact × effort × uncertainty × risk is false precision: six guesses multiplied into one confident-looking number, resolving real prioritization tension silently where the system should surface it. The correct behavior — user sets priority, attention states gate spend, genuine contention is *asked* about — is already specified under *Projects*. Reasoning in full there.
- **Constitution / Self-Modification Layer — absorbed.** Self-modification is an action and routes through the existing Authorization Framework (see *Self-modification is not exempt*). No separate constitutional stratum required.
- **Identity Layer — downsized to a field.** The distinction (verified vs claimed identity) is real and adopted; the "layer" framing is not. Lives on the People Graph, feeding the existing provenance gate.
- **Full belief-dependency graph ("Bazel/Nix for beliefs") — deferred indefinitely.** The achievable slice — derived facts recording their sources so reconciliation can cascade — is adopted under *Reconciliation*. A general truth-maintenance system that recomputes a belief network when any premise changes is a research problem, out of scope for a personal system's lifespan.

Further dispositions

- **Inferred emotional-state model — refused.** Reducing the user to detected states (overwhelmed / burned out / focused) and silently re-planning on that read is unreliable, invasive, and a manipulation surface. Replaced by a **user-declared focus mode** (see *World Model* → *User focus mode*). The refusal is the substance.
- **"Opportunity Sources" object type — folded, not added.** Environmental events (scholarship opens, CFP released, contact replies) feed the existing escalation machinery as an external subscription; an opportunity becomes a **candidate** project on the lifecycle axis. One mechanism pointed outward, not a new world-model object

(see *Projects* → *Environmental events*).

- **Expanded project-state list — resolved by separating axes.** The proposed `candidate` → `active` → `waiting` → `dormant` → `completed` → `archived` → `abandoned` conflated two independent dimensions. Resolved by splitting **lifecycle** (`candidate` | `active` | `completed` | `archived` | `abandoned`) from **attention** (`focus` | `background` | `dormant`), with "waiting" already covered by the temporal facet (see *Projects* → *Lifecycle*).

Out of scope: implementation choices, not architecture

The following are deliberately excluded from this document. They are real decisions, but they are *implementation* decisions that the object model above does not depend on, and admitting them here would only blur architecture with engineering:

- centralized vs distributed memory
- graph database vs relational store
- event bus vs direct calls
- internal planner algorithm
- model selection / routing

Each is chosen at build time against measured behaviour, not argued in a design doc. Naming them here is the treatment: scoped out on purpose.

On convergence. This design was built through repeated adversarial review. A pattern held across rounds: each pass surfaced roughly one thing the architecture genuinely could not express, wrapped in two-to-four additions it already could. The correct response was the same each time — absorb the real gap into an existing object, decline the rest. The signal that the architecture had stabilised was not the page count; it was that later rounds increasingly *removed* concepts (into existing structures) rather than *adding* them, and that the strongest closing advice any review offered was to distrust proposals beginning "add a new layer." The object model is stable. The remaining questions — whether the world model stays accurate, whether planning earns its complexity, whether the project object scales without becoming a catch-all — are implementation and research, not architecture. This is the stopping point.

Where This Meets Reality

Everything above this point has been architecture. Architecture is the cheap part — each new layer costs a paragraph to write and far more to build, and the gap between those two numbers is where systems like this usually stop being documents and start being abandoned. This section names the problems the design has **not** earned the right to call solved. They are not a to-do list; they are the honest edge of the napkin.

1. Reliable browser / computer control. Every capability that touches the real world routes through the Browser and Device agents. Today this kind of control is brittle: pages change, selectors break, vision-based control is slow and error-prone. The architecture assumes a reliability that does not yet exist off the shelf.

2. Durable memory that does not pollute. Provenance, expiration, and reconciliation are designed to fight memory rot, but they are mitigations, not a solution. Over months, a personal world model accumulates contradictions, stale facts, and low-confidence entries that quietly poison retrieval. Whether the lifecycle machinery actually holds the line over a year is unproven.

3. Accurate world models over long horizons. Related but distinct: not "does a fact go stale" but "does the *whole model* stay faithful to a changing life." Drift compounds. No one has a convincing answer for keeping a personal digital twin accurate over years without constant manual correction — which defeats the purpose.

4. Planning that actually beats a task queue. This is the existential one. The entire multi-agent planning apparatus is justified only if it produces materially better outcomes than a flat queue of tasks with good tools attached. If a simple queue captures most of the value, the Planner, the goal-dependency graph, and the project orchestration are overhead carried for a thin margin. A design artifact should state this plainly: the premise is unproven, and it is the first thing worth testing if this ever moves toward implementation.

5. Latency and cost. Plan → review → execute → verify, across multiple agents with independent verification, multiplies model calls per user action. The security and verification machinery that makes the system *safe* is the same machinery that makes it *slow and expensive*. Whether the result stays responsive and affordable enough to use daily is an open tradeoff, not a solved one.

The honest summary: the architecture is plausible. Plausible is not the same as proven, and the five problems above are where the proving would happen.